Universidad Nacional Autónoma de México

Ingeniería en ...

Nombre de Asignatura

FORMATO DE REPORTE

ALUMNO:
309075318

Grupo:
#
Semestre:
2024-II

México,CDMX. Febrero 2024

Índice

1. Introducción **2**

2. Marco Teórico **2**

3. Desarrollo **2**

3.1. Simulación de Reproductor de archivos de audio 2

3.1.1. Algoritmo 2

3.1.2. Descripción de Funciones 2

3.1.3. Pruebas 5

4. Resultados **5**

5. Conclusiones **5**

1. Introducción

Este apartado debe abordar los siguientes puntos:

- **Planteamiento del problema.** Se hace una descripción del problema a resolver.
- **Motivación.** Se describe porqué es necesario dar solución al problema.
- **Objetivos.** Lo que se se espera obtener de darle solución al problema.

2. Marco Teórico

Son los conceptos aplicados, principalmente los vistos en clase, con los que se va a dar solución al problema, explicados de forma concreta, ya sea con su interpretación personal o bien con referencias bibliográficas sólidas. [1]

3. Desarrollo

3.1. Simulación de Reproductor de archivos de audio

3.1.1. Algoritmo

1. Se muestra un menú de opciones.
2. Se registran nuevos usuarios en la estructura correspondiente según su tipo.
3. Se atienden los usuarios respetando la prioridad de los VIP sobre los usuarios normales.
4. Se muestran el contenido de ambas colas.

3.1.2. Descripción de Funciones

- Función **insertarCancion**: La función recibe dos argumentos: el apuntador doble a la lista de canciones y una cadena con el título de la canción. Su funcionamiento se divide en los siguientes pasos:

  1. Se crea un nuevo nodo, reservando dinámicamente memoria con la función **malloc**. Si no se pudo asignar memoria, se imprime un mensaje de error y la función termina.
  2. Se copia la cadena **titulo** al campo **nuevo->titulo** con ayuda de la función **strcpy**.
  3. Si la lista está vacía, el nuevo nodo se enlaza a sí mismo mediante **nuevo->siguiente = nueva** y el apuntador ***lista** se actualiza para apuntar al nuevo nodo.

4. Si la lista no está vacía, se utiliza un apuntador auxiliar **actual** para recorrer la lista hasta encontrar el último nodo, es decir, aquel cuyo campo **siguiente** apunta nuevamente al inicio de la lista.
5. El último nodo encontrado se enlaza con el nuevo nodo mediante **actual->siguiente = nueva**, y el nuevo nodo se enlaza al inicio de la lista con **nueva->siguiente = *lista**, manteniendo la estructura circular.
6. Finalmente, se imprime un mensaje indicando que la canción fue agregada a la lista.

- Función **buscarCancion**: La función recibe dos argumentos: el apuntador al inicio de la lista y una cadena con el título de la canción a buscar. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, la función regresa **NULL**.
  2. Se utiliza un apuntador auxiliar **actual** para comenzar el recorrido desde el primer nodo de la lista.
  3. Se recorre la lista circular con una estructura **do-while**, comparando en cada nodo el campo **titulo** con la cadena recibida, utilizando la función **strcmp**.
  4. Si se encuentra coincidencia, la función regresa el apuntador al nodo correspondiente.
  5. Si el recorrido termina y no se encontró la canción, la función regresa **NULL**.

- Función **eliminarCancion**: La función recibe tres argumentos: el apuntador doble a la lista, el apuntador doble a la canción en reproducción y una cadena con el título de la canción a eliminar. La función elimina el nodo correspondiente a la canción indicada. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, se imprime un mensaje y la función termina.
  2. Se utilizan dos apuntadores auxiliares: **actual**, que comienza en el inicio de la lista, y **anterior**, que inicialmente apunta a **NULL**.
  3. Se recorre la lista circular comparando el título de cada nodo con la cadena recibida, utilizando **strcmp**.
  4. Si se encuentra la canción, se distinguen tres casos:

     a) Si la lista solo tiene un elemento, se libera el nodo y el apuntador ***lista** se actualiza a **NULL**. Si esa canción era la que se estaba reproduciendo, también debe actualizarse ***reproduciendo** a **NULL**.

     b) Si el nodo a eliminar es el primero de una lista con más de un elemento, se busca el último nodo para actualizar su enlace hacia el nuevo inicio de la lista. Después se actualiza ***lista** al siguiente nodo.

   c) En el caso general, el nodo anterior se enlaza directamente con el siguiente nodo del actual, eliminando así el nodo encontrado de la lista.

5. Si la canción eliminada era la que se estaba reproduciendo, el apuntador ***reproduciendo** se actualiza al siguiente nodo.
6. Se libera la memoria del nodo eliminado con ayuda de la función **free()**.
7. Si la canción no se encuentra después de recorrer toda la lista, se imprime un mensaje indicando que no está en la lista.

- Función **empezarReproduccion**: La función recibe dos argumentos: el apuntador al inicio de la lista y el apuntador doble a la canción en reproducción. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, se imprime un mensaje, se asigna **NULL** a ***reproduciendo** y la función termina.
  2. Si la lista contiene canciones, el apuntador ***reproduciendo** se actualiza para apuntar al primer nodo de la lista.
  3. Finalmente, se imprime el título de la canción que comienza a reproducirse.

- Función **siguienteCancion**: La función recibe como argumento el apuntador doble a la canción en reproducción. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si existe una canción en reproducción. Si ***reproduciendo** == **NULL**, se imprime un mensaje y la función termina.
  2. El apuntador ***reproduciendo** avanza al siguiente nodo de la lista mediante el campo **siguiente**.
  3. Finalmente, se imprime el título de la nueva canción en reproducción.

- Función **mostrarLista**: La función recibe como argumento el apuntador al inicio de la lista circular. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, se imprime un mensaje y la función termina.
  2. Se utiliza un apuntador auxiliar **actual** para recorrer la lista desde el inicio.
  3. Mediante una estructura **do-while**, se imprime el título de cada canción y se avanza al siguiente nodo hasta regresar al inicio de la lista.

- Función **liberarLista**: La función recibe como argumento el apuntador doble a la lista circular. Su propósito es liberar la memoria dinámica ocupada por todos los nodos de la lista. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía. Si lo está, la función termina.

2. Se utiliza un apuntador auxiliar **actual** para comenzar el recorrido desde el nodo siguiente al primero.
3. Se recorre la lista liberando cada nodo hasta regresar al inicio.
4. Finalmente, se libera también el primer nodo y se asigna **NULL** a ***lista**.

3.1.3. Pruebas

Tipo	**Turno**	**Nombre**
VIP	1	Alan
Normal	1	Juan
Normal	2	Roberto
VIP	2	Raymundo
VIP	3	Diego

Tabla 1: Usuarios registrados en las colas

Turno	**Tipo**	**Nombre**
1	VIP	Alan
2	VIP	Raymundo
3	VIP	Diego
1	Normal	Juan
2	Normal	Roberto

Tabla 2: Orden de atención

4. Resultados

Mediante capturas de pantalla y una breve descripción seguida de la captura se presentan los resultados finales de su aplicación.

5. Conclusiones

Se presenta un análisis de los resultados obtenidos, donde se destaca la importancia de la aplicación de los conceptos teóricos para resolver el problema. Es importante resaltar lo siguiente:

- No es describir si les gustó la actividad o no.
- No es decir que se obtuvo de la práctica.
- No es describir lo que fue difícil.

Referencias

[1] Brian W. Kernighan y Dennis M. Ritchie. *The C Programming Language*. 2nd. Prentice Hall Professional Technical Reference, 1988. ISBN: 0131103709.